

35-Day Applied Machine Learning & ML Engineering Roadmap

Classical ML • Algorithms • Mathematics • Production Architecture • FastAPI • Docker
• Cloud

Target: Become capable of designing, training, evaluating, packaging, serving, and deploying practical ML systems.

Table of Contents

1. Roadmap Objective and Learning Philosophy
2. Starting Point and Assumptions
3. What You Will and Will Not Study
4. The ML Engineer Mental Model
5. Core ML Concepts and Mathematics
6. Supervised Learning Algorithms
7. Unsupervised Learning and Anomaly Detection
8. NLP with Classical Machine Learning
9. Evaluation, Validation, Leakage, and Model Selection
10. Feature Engineering, Selection, and Explainability
11. Hyperparameter Optimization
12. Production ML Architecture
13. Pipelines, Experiment Tracking, and Model Lifecycle
14. Model Serving with FastAPI
15. Docker and Cloud Foundations
16. The 35-Day Study Plan
17. Project Portfolio
18. ML Model Arena
19. Algorithm Decision Guide
20. Production Checklist
21. What to Learn After Day 35

1. Roadmap Objective and Learning Philosophy

This roadmap is designed for someone who already knows Python, SQL, basic NumPy/Pandas, basic preprocessing, and the idea of linear and logistic regression through scikit-learn. The goal is not to spend a month repeating EDA or memorizing sklearn APIs. The goal is to build a strong classical/applied ML foundation and then turn that knowledge into an ML engineering workflow.

The target

- Understand the mathematics and intuition behind the core ML algorithms.
- Know when to use a model, when not to use it, and what assumptions it makes.
- Build reliable training/validation/test strategies.
- Prevent data leakage and understand overfitting and generalization.
- Compare models using appropriate metrics rather than chasing accuracy.
- Build reusable preprocessing + model pipelines.
- Track experiments and model artifacts.
- Version and register models at a basic practical level.
- Expose a trained pipeline through FastAPI.
- Containerize the service with Docker.
- Deploy one basic ML service to a cloud environment.
- Think in terms of complete ML systems rather than isolated notebooks.

The key mindset shift

Junior-style ML often looks like:

```
data → model.fit() → model.predict() → accuracy
```

The target workflow is:

```
Problem definition  
→ data/query strategy  
→ split strategy  
→ baseline  
→ preprocessing + feature engineering  
→ candidate models  
→ cross-validation  
→ hyperparameter tuning  
→ error analysis  
→ final pipeline  
→ experiment tracking  
→ model artifact/version  
→ API inference  
→ container  
→ deployment  
→ monitoring considerations
```

2. Starting Point and Assumptions

You are assumed to already be comfortable with:

- Python syntax, functions, loops, imports, and basic object-oriented concepts.
- SQL queries and relational data concepts.
- Basic NumPy array operations.

- Basic Pandas loading, selection, modification, and missing-value handling.
- Basic preprocessing: nulls, zeros where appropriate, outliers, encoding, scaling, and transformations.
- Basic scikit-learn usage.
- LinearRegression and LogisticRegression as library models.

You have not yet developed strong intuition for gradient descent, ML mathematics, model internals, or production ML architecture. Those are deliberate priorities in this roadmap.

3. What You Will and Will Not Study

High-priority topics

- Linear and logistic regression in depth.
- Ridge, Lasso, Elastic Net.
- Polynomial regression.
- KNN.
- Decision trees.
- Random forests.
- SVM.
- Naive Bayes.
- Gradient boosting.
- XGBoost, LightGBM, and CatBoost at practical depth.
- K-Means and hierarchical clustering.
- PCA.
- Isolation Forest, One-Class SVM, and LOF.
- Classical NLP: Bag of Words, TF-IDF, n-grams.
- Evaluation and validation.
- Feature engineering and feature selection.
- Hyperparameter tuning.
- sklearn Pipeline and ColumnTransformer.
- MLflow basics.
- Model persistence and lifecycle.
- FastAPI serving.
- Docker basics.
- Cloud deployment basics.

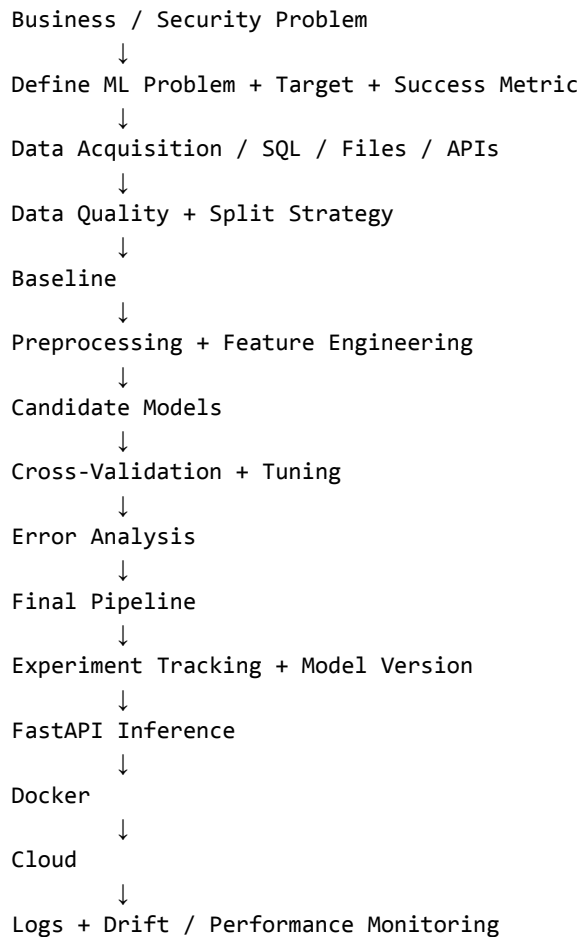
Intentionally outside the 35-day core

- Deep learning, CNNs, RNNs, Transformers, and LLM fine-tuning.
- Kubernetes and complex orchestration.
- Distributed training.
- Advanced cloud architecture.
- Advanced MLOps platforms.
- Trying to deeply master every possible ML algorithm.

4. The ML Engineer Mental Model

Every ML problem should be viewed as a system. A model is one component of that system.

End-to-end architecture



Ten questions for every algorithm

22. What problem does it solve?
23. What is the mathematical objective?
24. What assumptions does it make?
25. How does training work?
26. What parameters does it learn?
27. Which hyperparameters control it?
28. Why can it overfit?
29. How can overfitting be reduced?
30. When should it not be used?
31. What changes when it is used in production?

5. Core ML Concepts and Mathematics

5.1 Linear algebra

- Scalars, vectors, matrices, tensors as a conceptual extension.
- Shapes and dimensions.
- Dot product and matrix multiplication.
- Transpose.
- Linear combinations and feature representation.
- Matrix inverse and why it matters for the normal equation.
- Covariance matrices for PCA.

5.2 Probability and statistics

- Probability and conditional probability.
- Bayes theorem.
- Expectation and variance.
- Distributions and sampling intuition.
- Correlation and covariance.
- Statistical uncertainty and why a test score is not absolute truth.

5.3 Derivatives and gradients

A derivative describes how a quantity changes. A gradient generalizes this idea to multiple parameters.

Gradient descent: $\theta_{\text{new}} = \theta_{\text{old}} - \alpha \nabla J(\theta)$

- θ : learnable parameters.
- α : learning rate.
- $J(\theta)$: objective/loss function.
- $\nabla J(\theta)$: direction of greatest increase; subtracting it moves toward lower loss.

5.4 Gradient descent

- Batch gradient descent: uses the full training set for an update.
- Stochastic gradient descent: updates from individual examples.
- Mini-batch gradient descent: practical compromise used widely in ML.
- Learning-rate too small: slow convergence.
- Learning-rate too large: unstable or divergent optimization.
- Feature scaling can substantially affect optimization.

5.5 Loss functions

- MSE for regression.
- MAE for a more robust absolute-error objective.
- Log loss / binary cross entropy for probabilistic classification.
- Regularized objectives combine data-fit loss with a complexity penalty.

6. Supervised Learning Algorithms

6.1 Linear Regression

- Model: $y_{\text{hat}} = Xw + b$.

- Understand coefficients, intercept, residuals, SSE/RSS, MSE, MAE, RMSE, R^2 , and adjusted R^2 .
- Normal equation: $w = (X^T X)^{-1} X^T y$ when the relevant assumptions and invertibility conditions are satisfied.
- Gradient-descent training and why scaling can matter.
- Assumptions: linear relationship, useful residual behavior, limited problematic multicollinearity, and appropriate error structure.
- Failure modes: nonlinear relationships, severe outliers, strong multicollinearity, and leakage.

6.2 Polynomial Regression

- Generate polynomial features while keeping a linear estimator.
- Understand model capacity and why higher degree can overfit.
- Use regularization and validation to control complexity.

6.3 Ridge, Lasso, and Elastic Net

- Ridge adds an L2 penalty and tends to shrink coefficients.
- Lasso adds an L1 penalty and can drive coefficients to zero, supporting feature selection.
- Elastic Net combines L1 and L2 behavior.
- Understand λ/α as a complexity-control hyperparameter.

6.4 Logistic Regression

- Linear score $z = Xw + b$ followed by a sigmoid probability.
- Sigmoid: $p = 1 / (1 + \exp(-z))$.
- Log-odds: $\log(p/(1-p)) = Xw + b$.
- Binary cross-entropy/log loss.
- Decision thresholds are separate from probability estimation.
- Multiclass strategies and regularization.
- Understand why logistic regression can be an excellent baseline.

6.5 K-Nearest Neighbors

- Prediction is based on nearby training observations.
- Distance metrics: Euclidean, Manhattan, and others.
- K controls smoothness and bias/variance.
- Scaling is critical because distance is scale-sensitive.
- Prediction can become expensive as the training set grows.

6.6 Decision Trees

- Recursive partitioning of feature space.
- Entropy and information gain.
- Gini impurity.
- Maximum depth, minimum samples, pruning concepts.
- Trees naturally model nonlinear interactions but can overfit.

6.7 Random Forest

- Bagging plus randomized feature selection.
- Bootstrap samples create diverse trees.
- Aggregation reduces variance.
- Out-of-bag evaluation concept.

- Feature importance: understand impurity importance and permutation importance.

6.8 Support Vector Machines

- Find a separating hyperplane with a large margin.
- Support vectors define the boundary.
- C controls the tradeoff between margin and training errors.
- Kernel trick allows nonlinear decision boundaries.
- RBF kernel and gamma intuition.
- Scaling is generally important.

6.9 Naive Bayes

- Bayes theorem with a conditional-independence assumption.
- Gaussian, Multinomial, and Bernoulli variants.
- Very useful as a fast baseline for classical text classification.
- Understand why a simplifying assumption can still work well.

6.10 Gradient Boosting

- Sequentially add weak learners that improve the current ensemble.
- Understand residual/error correction and gradient-based loss minimization.
- Learning rate and number of estimators trade off with each other.
- Tree depth controls learner complexity.

6.11 XGBoost / LightGBM / CatBoost

- Learn the common gradient-boosting ideas first.
- Then learn practical differences: speed, categorical handling, regularization, tree growth strategies, and typical use cases.
- Focus on hyperparameter reasoning rather than memorizing every parameter.

7. Unsupervised Learning and Anomaly Detection

7.1 K-Means

- Choose K centroids.
- Assign each observation to its nearest centroid.
- Update centroids.
- Repeat until convergence.
- Understand inertia, elbow method, initialization, scaling, and limitations.

7.2 Hierarchical Clustering

- Agglomerative clustering starts with individual observations.
- Merge clusters according to a linkage rule.
- Understand single, complete, average, and Ward linkage at a practical level.
- Read dendrograms to choose a cut level.

7.3 PCA

- Find directions of maximum variance.
- Connect covariance matrices to eigenvectors/eigenvalues.

- Understand explained variance ratio.
- Project observations into a lower-dimensional space.
- Use PCA for compression, visualization, noise reduction, or preprocessing—not as a universal improvement.

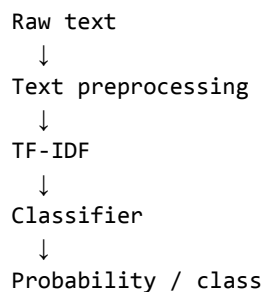
7.4 Anomaly detection

- Isolation Forest: isolate unusual observations efficiently.
- One-Class SVM: learn a boundary around normal data.
- Local Outlier Factor: compare local density.
- Choose methods according to data geometry, scale, and operational constraints.

8. NLP with Classical Machine Learning

The focus is text as structured features, not neural language models.

- Tokenization concepts.
- Bag of Words.
- Vocabulary and sparse matrices.
- TF-IDF.
- Unigrams and n-grams.
- Text normalization.
- Multinomial Naive Bayes.
- Logistic Regression.
- Linear SVM.
- Class imbalance and threshold selection.



Portfolio direction: spam/phishing classification, sentiment or support-ticket routing, and other classical text classification tasks.

9. Evaluation, Validation, Leakage, and Model Selection

9.1 Regression metrics

- MAE
- MSE
- RMSE
- R^2
- Adjusted R^2
- MAPE with awareness of its limitations

9.2 Classification metrics

- Confusion matrix.
- Accuracy.
- Precision.
- Recall/sensitivity.
- Specificity.
- F1 score.
- ROC-AUC.
- Precision-Recall AUC.
- Log loss.
- Calibration and probability quality.

9.3 Thresholds

A classifier's probability estimate and its final class decision are different layers. A threshold of 0.5 is a convention, not a law.

9.4 Cross-validation

- K-fold cross-validation.
- Stratified K-fold for classification.
- Time-series splits when temporal ordering matters.
- Validation design must respect how the model will encounter data in production.

9.5 Data leakage

- Scaling or imputation using the entire dataset before splitting.
- Using future information to predict the past.
- Target leakage through derived features.
- Duplicate records appearing across train and validation/test.
- Human-generated labels or post-outcome information leaking into features.
- Performing feature selection on all data before cross-validation.

9.6 Bias and variance

- Underfitting: model is too simple to capture useful structure.
- Overfitting: model learns training-specific patterns that do not generalize.
- Bias-variance tradeoff.
- Regularization, validation, model complexity, and dataset size all affect generalization.

10. Feature Engineering, Selection, and Explainability

10.1 Feature engineering

- Ratios and rates.
- Interactions.
- Aggregations by entity.
- Temporal features.
- Log and power transformations.

- Binning where justified.
- Domain-specific security and financial signals.
- Text-derived sparse features.

10.2 Feature selection

- Filter methods: correlation, mutual information, chi-square.
- Wrapper methods: RFE.
- Embedded methods: Lasso and tree-based importance.
- Permutation importance.

10.3 Explainability

- Global feature importance.
- Local explanation: why one prediction was made.
- Permutation importance.
- SHAP as a later-stage practical tool.
- Always distinguish correlation/importance from causation.

11. Hyperparameter Optimization

- Manual tuning for intuition.
- GridSearchCV for small, focused search spaces.
- RandomizedSearchCV for broader spaces.
- Successive-halving concepts.
- Bayesian optimization concept: use previous evaluations to choose promising future trials.

Parameter vs hyperparameter

A parameter is learned from data during training, such as regression coefficients. A hyperparameter is selected outside the model's parameter-learning process, such as tree depth, K in KNN, or regularization strength.

Tuning discipline

- Define the evaluation metric first.
- Keep the final test set untouched until the end.
- Use pipelines so preprocessing is included inside validation.
- Track experiments rather than relying on memory.
- Tune only parameters that can plausibly change the model's behavior.

12. Production ML Architecture

12.1 Training system vs inference system

TRAINING

Data → validation strategy → preprocessing → features → model training → evaluation → artifact

INFERENCE

Request → input validation → same preprocessing/features → model → prediction → response

The most important production rule is that training and inference must apply compatible transformations. A serialized sklearn Pipeline is one practical way to keep preprocessing and prediction logic together.

12.2 Recommended project structure

```
ml-project/
├── data/
├── notebooks/
├── src/
│   ├── data/
│   ├── features/
│   ├── models/
│   ├── training/
│   └── evaluation/
├── api/
├── tests/
├── configs/
├── models/
├── logs/
├── scripts/
├── requirements.txt
└── README.md
```

12.3 Separation of concerns

- Notebook: exploration and experiments.
- src/data: acquisition/loading and data checks.
- src/features: reusable feature logic.
- src/models: model definitions or construction.
- src/training: orchestration of training.
- src/evaluation: metrics and error analysis.
- api: inference service.
- tests: verify critical behavior.
- configs: parameters and environment-specific settings.

13. Pipelines, Experiment Tracking, and Model Lifecycle

13.1 sklearn Pipeline

```
Raw features
  ↓
ColumnTransformer
  ↓
Feature transformations
  ↓
Estimator
  ↓
Prediction
```

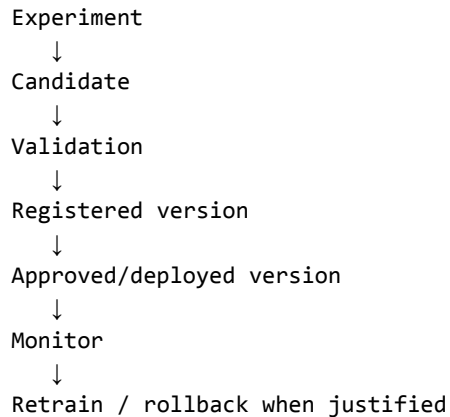
This reduces training/inference mismatch and makes cross-validation safer because transformations can be fitted inside each training fold.

13.2 MLflow

- Experiment: a logical group of runs.

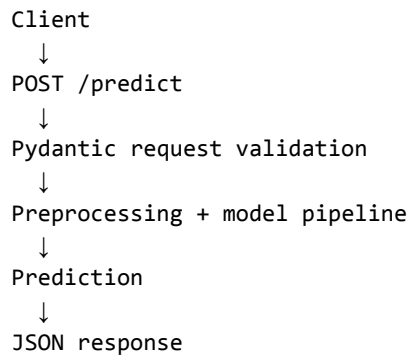
- Run: one training attempt.
- Parameters: configuration values.
- Metrics: measured outcomes.
- Artifacts: files such as models, plots, reports, or datasets metadata.
- Model: serialized model/pipeline artifact.
- Registry: model versions and lifecycle metadata.

13.3 Model lifecycle



14. Model Serving with FastAPI

14.1 Inference architecture



14.2 Useful endpoints

- GET /health — service health check.
- GET /model-info — basic model/version metadata.
- POST /predict — inference endpoint.

14.3 Example request/response

```

POST /predict
{
  "age": 32,
  "income": 85000,
  "transactions": 17
}
  
```

Response:

```
{
  "prediction": 1,
  "probability": 0.93
}
```

14.4 API concerns

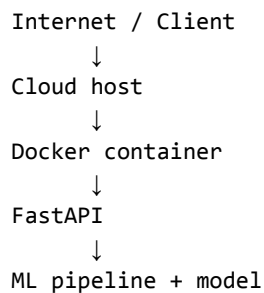
- Input validation.
- Schema stability.
- Error handling.
- Logging.
- Latency.
- Model loading strategy.
- Dependency compatibility.
- Health checks.

15. Docker and Cloud Foundations

15.1 Docker concepts

- Image: immutable package containing the application and runtime environment.
- Container: running instance of an image.
- Dockerfile: instructions for building an image.
- Port mapping: exposes a container service.
- Environment variables: runtime configuration.
- .dockerignore: prevents unnecessary files entering the build context.

15.2 ML service container



15.3 Cloud basics

- Compute: where the service runs.
- Storage: where artifacts/data can live.
- Networking: how requests reach the service.
- Environment variables and secrets.
- Identity/access control concepts.
- Logs and service health.

Kubernetes, distributed systems, advanced IAM, CI/CD platforms, and multi-region architecture are deliberately postponed.

16. The 35-Day Study Plan

Baseline schedule: approximately 5 hours/day of structured learning, plus project time. Do not treat every day as a one-topic lesson; several days deliberately combine related concepts.

Day	Learning focus	Practical output
Day 1 — ML Foundations + Mathematics	ML terminology, supervised vs unsupervised, parameters vs hyperparameters, vectors, matrices, dot products, derivatives, gradients, optimization.	Project 1: implement the mathematics of linear regression with NumPy.
Day 2 — Linear Regression	MSE, normal equation, gradient descent, coefficients, intercept, residuals, assumptions.	House-price regression; compare normal equation and gradient descent.
Day 3 — Regression Evaluation + Validation	MAE, MSE, RMSE, R^2 , adjusted R^2 , train/test split, validation, cross-validation.	Business revenue prediction + metric comparison.
Day 4 — Polynomial Regression + Regularization	Polynomial features, bias/variance, Ridge, Lasso, Elastic Net.	Demand prediction comparing linear, polynomial, Ridge, Lasso, Elastic Net.
Day 5 — Logistic Regression	Sigmoid, probability, odds/log-odds, log loss, decision boundary, gradient descent.	Customer churn prediction.
Day 6 — Classification Evaluation	Confusion matrix, precision, recall, F1, specificity, ROC-AUC, PR-AUC, threshold tuning.	Fraud detection; demonstrate why accuracy can be misleading.
Day 7 — Week 1 Combined	End-to-end supervised learning workflow.	Credit-risk prediction: preprocessing → baseline → model → validation → metrics → error analysis.
Day 8 — Decision Trees	Entropy, information gain, Gini impurity, splits, depth, pruning, overfitting.	Customer classification.
Day 9 — Random Forest	Bagging, bootstrap sampling, randomized features, OOB evaluation, feature importance.	Network intrusion classification.
Day 10 — KNN + SVM	Distance metrics, K, scaling, hyperplanes, margins, support vectors, C, kernels.	Security-event classification; compare multiple models.
Day 11 — Naive Bayes + Classical NLP	Bayes theorem, conditional independence, Bag of Words, TF-IDF, n-grams.	Spam/phishing text classifier.
Day 12 — Gradient Boosting	Weak learners, residual correction, loss minimization, learning rate, estimator count.	Financial-risk prediction.
Day 13 — XGBoost / LightGBM / CatBoost	Practical gradient-boosting families, hyperparameters, categorical handling, regularization.	Serious tabular model comparison.

Day 14 — Week 2 Combined	Model comparison and cybersecurity workflow.	Cybersecurity threat detection: baseline → RF → boosting → tuning → error analysis.
Day 15 — K-Means	Centroids, assignment/update loop, inertia, elbow method, initialization.	Customer segmentation.
Day 16 — Hierarchical Clustering	Agglomerative clustering, linkage, dendrograms.	Business customer segmentation.
Day 17 — PCA	Covariance, eigenvectors/eigenvalues, explained variance, projection.	Dimensionality reduction + downstream model.
Day 18 — Anomaly Detection	Isolation Forest, One-Class SVM, LOF, anomaly scores.	Cybersecurity anomaly detection.
Day 19 — Feature Engineering	Interactions, ratios, aggregations, temporal and domain features, transformations.	Improve a weak model without changing the algorithm.
Day 20 — Feature Selection + Explainability	Mutual information, chi-square, RFE, Lasso, permutation importance, SHAP concepts.	Explain model decisions and reduce unnecessary features.
Day 21 — Week 3 Combined	Supervised + unsupervised workflow.	Financial anomaly/risk detection combining feature engineering, PCA, clustering, anomaly detection, and supervised ML.
Day 22 — Validation Architecture	Train/validation/test, K-fold, stratification, time splits, leakage, baselines.	Intentionally introduce and then fix leakage in a previous project.
Day 23 — Hyperparameter Optimization	Grid search, randomized search, successive-halving concepts, Bayesian optimization concept.	Tune the best model from an earlier project.
Day 24 — sklearn Pipelines	Pipeline, ColumnTransformer, transformation/model composition.	Convert an old notebook into a reproducible training pipeline.
Day 25 — ML Project Architecture	Project structure, modules, configuration, logging, testing, separation of training and inference.	Refactor a project into an engineering-grade repository.
Day 26 — Experiment Tracking	MLflow experiments, runs, parameters, metrics, artifacts, model logging.	Track multiple model experiments.
Day 27 — Model Registry + Lifecycle	Model versions, lineage, metadata, aliases, approval and rollback concepts.	Register and version the best model.
Day 28 — Model Serving	Serialization, model loading, FastAPI, request validation, inference.	Create /predict, /health, and /model-info.
Day 29 — Complete ML API	API testing, errors, logging, latency, schema stability.	Turn a trained pipeline into a usable inference service.
Day 30 — 30-Day ML System Checkpoint	End-to-end system architecture.	Training → validation → MLflow → model artifact → FastAPI.
Day 31 — Docker Fundamentals	Image, container, Dockerfile, ports, environment variables, .dockerignore.	Containerize the FastAPI ML service.

Day 32 — Docker + ML	Production-oriented container configuration, health checks, runtime dependencies.	Run and test the complete ML container.
Day 33 — Cloud Fundamentals	Compute, storage, networking, secrets, logs, service exposure.	Choose a simple deployment target and prepare the service.
Day 34 — Cloud Deployment	Deploying a containerized FastAPI ML service.	Put one ML API online and test it.
Day 35 — Final ML Engineer Project	Integrate algorithms, evaluation, pipelines, tracking, serving, Docker and cloud.	Finish one flagship system and polish the portfolio.

17. Project Portfolio

The final portfolio should contain three strong projects, not dozens of disconnected notebooks.

Project 1 — SentinelML: Cybersecurity Threat Detection

Goal: classify malicious/suspicious network activity.

Models: Logistic Regression, Random Forest, XGBoost/LightGBM/CatBoost.

Engineering: reproducible pipeline, validation, MLflow tracking, model versioning, FastAPI, Docker, basic cloud deployment.

Metrics: precision, recall, F1, ROC-AUC, PR-AUC; explain why the chosen metric matches security costs.

Project 2 — FinSight: Financial / Business Risk Engine

Goal: estimate risk or customer outcome from financial/business features.

Models: Logistic Regression, Random Forest, Gradient Boosting/XGBoost.

Engineering: probability output, threshold selection, calibration concepts, feature importance, error analysis, API.

Business focus: explain the model in terms of cost, risk, and decision thresholds rather than only model score.

Project 3 — TextGuard: Classical NLP Text Classifier

Goal: classify phishing/spam or another realistic text category.

Pipeline: text preprocessing → TF-IDF → Logistic Regression / Naive Bayes / Linear SVM.

Engineering: sklearn Pipeline, sparse features, FastAPI inference, Docker.

Demonstrates: unstructured text + classical ML + production serving without deep learning.

18. ML Model Arena

Build a reusable mini-framework that trains several candidate models under the same validation protocol and produces a comparison table.

Model	F1	ROC-AUC	Training Time
Logistic	0.82	0.89	0.12s
Random Forest	0.88	0.94	1.42s
SVM	0.85	0.92	2.21s
XGBoost	0.91	0.96	0.81s
CatBoost	0.90	0.95	1.10s

- Use the same train/test protocol.
- Use cross-validation on the training data.
- Keep the final test set untouched.
- Record training time and model size where useful.
- Compare metrics relevant to the problem.
- Record the final choice and its rationale.

19. Algorithm Decision Guide

Situation	Strong starting candidates	Why
Simple tabular regression	Linear Regression / Ridge	Fast, interpretable baseline.
Nonlinear tabular classification	Random Forest / Gradient Boosting	Captures nonlinear interactions.
Strong tabular performance	XGBoost / LightGBM / CatBoost	Powerful gradient-boosted trees.
High-dimensional sparse text	Logistic Regression / Linear SVM / Naive Bayes	Efficient with sparse TF-IDF features.
Small dataset with smooth boundaries	SVM / KNN	Can work well when scaling and geometry are appropriate.
Customer segmentation	K-Means / Hierarchical	Useful exploratory grouping without labels.
Dimensionality reduction	PCA	Compresses correlated numeric features.
Unknown anomalies	Isolation Forest / LOF / One-Class SVM	Learn unusualness without ordinary class labels.
Need interpretable baseline	Linear/Logistic Regression / shallow tree	Simple behavior and explainability.

20. Production Checklist

- ☐ Problem is clearly defined.
- ☐ Target and prediction horizon are defined.
- ☐ Success metric is connected to business/security cost.
- ☐ Data source and assumptions are documented.
- ☐ Train/validation/test strategy is appropriate.
- ☐ Leakage risks have been checked.
- ☐ A simple baseline exists.
- ☐ Preprocessing is reproducible.
- ☐ Feature engineering is reproducible.
- ☐ Model comparison is based on validation data.
- ☐ Hyperparameters were tuned without contaminating the test set.
- ☐ Final test set was used only for final assessment.

- ☐ Errors and false positives/negatives were analyzed.
- ☐ Final preprocessing + estimator are packaged together.
- ☐ Experiment metadata is tracked.
- ☐ Model artifact has a version.
- ☐ Inference input schema is validated.
- ☐ API has health and prediction endpoints.
- ☐ Logs are available.
- ☐ Docker image builds and runs reproducibly.
- ☐ Cloud deployment is tested.
- ☐ Model and data drift risks are documented.
- ☐ Retraining/rollback criteria are at least conceptually defined.

21. What to Learn After Day 35

After this roadmap, the next step should be depth rather than collecting more algorithms.

- Advanced MLOps: CI/CD, automated retraining, feature stores, artifact stores, observability.
- Cloud ML: managed training, managed model endpoints, object storage, IAM, networking.
- Advanced model monitoring: drift detection, calibration monitoring, data quality checks, delayed-label evaluation.
- Time-series ML.
- Recommendation systems.
- Causal inference.
- Advanced Bayesian methods.
- Deep learning when you are ready for it.
- Modern AI/LLM systems after the classical ML foundation is solid.
- System design for ML: data contracts, online/offline features, batch inference, streaming inference, scaling and reliability.

Final outcome

The practical target is not to claim mastery of every ML technique. The target is to become capable of taking a real tabular, cybersecurity, finance/business, or classical NLP problem and moving it from problem definition through model selection, validation, feature engineering, experiment tracking, packaging, API serving, Docker, and basic cloud deployment—with the ability to explain the mathematics, assumptions, tradeoffs, and failure modes of the major algorithms used.

Current Ecosystem Notes

The roadmap intentionally emphasizes reproducible pipelines, cross-validation, experiment tracking, model versioning, and serving because these are core pieces of modern applied ML workflows. For current reference documentation, use the official scikit-learn documentation for model selection and pipelines, and MLflow documentation for experiment tracking and model registry concepts.

- scikit-learn: model selection, cross-validation, pipelines, preprocessing, metrics.
- MLflow: tracking, artifacts, model logging, model registry and lifecycle.
- FastAPI: API design and request validation.
- Docker: containerization and reproducible runtime environments.